

ROOTS-AI™ — M2 Assessment & Scoring: Completion Package

Milestone: M2 — Assessment & Scoring

Controlled sources: C-01 Canonical Question Bank v1.0.1 CORRECTED · C-02 Canonical Scoring Rules & Golden Tests v1.0.1 CORRECTED

Build / commit: ROOTS-AI-Health-Systems/demo-repository · branch `main` · commit `4056363` · **Deployment for**

verification: <https://root-ai-updated.vercel.app>

Prepared: 22 September 2026

This is the single consolidated M2 submission requested in item 12. Each section below maps one M2 checklist item to its implementation, its test and its evidence, with the command to reproduce it.

Summary of results

#	M2 requirement	Result	Evidence
1	Complete canonical assessment — 73 / 13 / 71 / 2 / 40	PASS	<code>evidence/automated-tests.txt</code> (source suite)
2	Validation (all C-01 rules)	PASS	<code>evidence/automated-tests.txt</code> (validation + bypass suites)
3	Assessment progress across 13 modules	PASS	<code>evidence/automated-tests.txt</code> (progress suite)
4	Deterministic seven-domain scoring (C-02 v1.0.1)	PASS	Golden Tests + source suite
5	Driver logic (DRV-001...004 v1.0.1)	PASS	Golden Tests + driver suite
6	No AI authority over scoring	PASS	\$6 below
7	Versioning (C-01 / C-02 v1.0.1 CORRECTED persisted)	PASS	versioning tests + DB tests BR-14, IN-07, SCH-01
8	30 Golden Tests	30/30 PASS	<code>evidence/golden-tests.pdf</code> / <code>.html</code> / <code>.csv</code> / <code>.json</code>
9	Security and negative tests	73/73 DB · 50/50 API · 23/23 bypass	<code>evidence/db-security-tests.live.csv</code> , <code>evidence/api-negative-tests.html</code> , <code>ROOTS-AI_M2_Security_Tests.sql</code>
10	Controlled technical source (XLSX → generated data)	PASS	<code>evidence/source-check.txt</code>
11	Code pushed to ROOTS GitHub	PASS	<code>ROOTS-AI-Health-Systems/demo-repository</code> @ <code>4056363</code>
12	One consolidated package	This document	—

1. Complete canonical assessment

- **Implementation.** `scripts/canonical/generate.py` reads `02_ROOTS_AI_C01_Canonical_Question_Bank_v1.0.1_CORRECTED.xlsx` and writes `lib/assessment/c01-question-bank.json`. The application loads questions, modules, option sets and validation messages only from that file; nothing is retyped. Question IDs, wording, order, module assignment, response types, option sets, scoring eligibility and Phase 1 visibility are exactly as in C-01.
- **Tests** (`tests/scoring/source.test.ts`): exactly 73 questions Q1...Q73 in order; 13 modules ordered 1–13; 71 required; 2 optional (Q5, Q73); 40 scoring-eligible, identical to the C-02 mapping; every C-01 QA check in the workbook is PASS.

2. Validation

- **Implementation.** `lib/assessment/validation.ts` — server-authoritative; the browser runs the same functions for immediate feedback. Error wording is read from the C-01 Validation sheet (VAL-001...011).
- **Behaviour.**
 - A required question is complete only with a valid response, or an explicit N/A where C-01 permits it.
 - Q13, Q14, Q52, Q53, Q54: at least one approved option; **an empty array is invalid**.
 - NONE / N/A are mutually exclusive with other selections.
 - Q73 is optional free text, 0–1,000 characters, **no N/A**.
- **Missing is never treated as N/A.**
- Submission re-validates **every stored answer on the server** (`checkStoredAnswers`), so a value written around the API cannot be submitted or scored.
- "Next" is governed: required questions on a module must be answered before moving on (C-05 ASM-06).
- **Tests.** `tests/scoring/source.test.ts` (validation suite) and `tests/security/bypass.test.ts` (14 tampering cases blocked at submission).

3. Assessment progress

- **Implementation.** `lib/assessment/progress.ts` — progress is computed from the saved answers on the server (the 71 required questions), never from a client value; 13 module states (complete / needs attention / not started); resume returns to the first module with an unanswered required question. Autosave/resume from M1 is unchanged.
- **Tests.** `tests/assessment/progress.test.ts` — 0% → 100%; optional questions neither block nor add; monotonic; unknown/duplicate IDs cannot inflate progress; all 13 module states; resume point; identical results after refresh / signing in again.

4. Deterministic seven-domain scoring

- **Implementation.** `lib/scoring/engine.ts` (pure function, exact integer arithmetic) with `lib/scoring/c02-ruleset.ts`, whose tables are read from `lib/scoring/c02-ruleset.json`, generated from `03_ROOTS_AI_C02_Canonical_Scoring_Rules_and_Golden_Tests_v1.0.1_CORRECTED.xlsx`. Scoring runs only on the server, at submission, from the stored answers.
- **Canonical rules implemented as written:** 40 scoring-eligible items only; raw burden scale 0–4; Q26/Q28 reverse-scored; 50% domain coverage threshold (below → null); Biological State requires ≥ 5 of 7 domains (else null);

integer rounding half-away-from-zero; Opportunity and Recovery Potential to one decimal; weights, thresholds, classifications, N/A and missing handling, coverage, confidence and evidence per C-02.

- **Tests.** Every formula constant held in code is checked against the workbook's own rule text (`tests/scoring/source.test.ts`); contextual (non-scoring) answers are proven not to change any score (`tests/security/bypass.test.ts`).

5. Driver logic

- **Implementation (DRV-001...004 v1.0.1).** Only available domains scoring ≥ 25 are eligible; ranking by score descending; exact ties in the fixed order **MR** $\rightarrow$ **HS** $\rightarrow$ **SR** $\rightarrow$ **CH** $\rightarrow$ **SL** $\rightarrow$ **IB** $\rightarrow$ **BS** (parsed from the DRV-002 rule text); if the top two eligible scores differ by ≤ 3 they are reported **once** as a co-primary pair followed by the next distinct eligible domain — no duplication, no ineligible or null fallback; no eligible domain $\rightarrow$ no driver. `BIO_STATE` denotes Biological State; `BS` is exclusively Biological Safety Signals™.
- **Tests.** All 30 Golden Tests; and for all 30 cases, drivers are eligible, distinct and never refer to `BIO_STATE` (`tests/scoring/source.test.ts`).

6. No AI authority over scoring

The M2 build contains **no AI/LLM component**: no AI library, API call or model is used anywhere in the application. Scores, classifications, Biological State, drivers, thresholds, eligibility, missing-data behaviour and Golden Test outputs are produced solely by the deterministic engine. The engine is a pure function (no I/O, clock, randomness or network); stored score rows cannot be changed afterwards by any role (DB test IN-01), and no API route accepts scores, drivers or classifications from a client (`tests/security/bypass.test.ts`).

The AI narrative layer is an **M3** deliverable. When it is added it will be read-only with respect to all deterministic outputs, with the M3 AI-boundary evidence (malformed output, score injection, prohibited language, timeout/retry/fallback).

7. Versioning

- **What is persisted.**
- Every assessment records, when it starts, the controlled question bank it is answered against: `{ "c01": { "label": "C-01 v1.0.1 CORRECTED", "questionnaire_version": "1.0.0", "document", "sha256" } }`.
- Every score row records the question bank and the scoring rules it was calculated from — `C-01 v1.0.1 CORRECTED` and `C-02 v1.0.1 CORRECTED`, `scoring_version 1.0.1`, dataset IDs and both workbook SHA-256 fingerprints — also inside its calculation trace.
- Each answer is tied to its assessment, and so to the same source.
- **Protection.** The source record is written by the server only, once, and can never be changed — not by the participant, not by staff, not by the server itself (DB tests BR-14, IN-07, SCH-01).
- **Note on the two version identifiers.** The C-01 v1.0.1 CORRECTED workbook states `questionnaire_version = 1.0.0` in its README and data rows. We store that value exactly as the workbook states it, and store the controlled document identity (`C-01 v1.0.1 CORRECTED` + SHA-256) alongside it, so the two are never conflated. Please confirm this is the intended reading.

8. 30 Golden Tests — 30/30 PASS

- **Evidence.** `evidence/golden-tests.pdf` (also `.html`, `.csv`, `.json`): for every case **Test ID** $\rightarrow$ **Input** $\rightarrow$ **Expected Output** $\rightarrow$ **Actual Output** $\rightarrow$ **PASS/FAIL**, with a field-by-field comparison of the eight canonical output fields (exact

equality, no tolerance), including corrected driver behaviour, null behaviour, tie / co-primary behaviour and rounding.

- **Supplementary classification check (30/30).** The canonical expected output does not list classifications, so every domain, confidence and recovery label is also compared with an independent lookup in the workbook's Classifications sheet.
- **Reproduce.** `npm ci` → `npm run evidence:golden` (regenerates the report) · `npm run test:scoring` (the same 30 cases as automated tests).
- **Note on one boundary rule.** Classification bands are whole-number ranges while Recovery Potential has one decimal. A value takes the highest band whose minimum it has reached (74.5 → Moderate, 75.0 → High). No Golden Test falls on such a boundary. Please confirm.

9. Security and negative tests

1. **Database / RLS — 73/73 PASS on the live database** (`evidence/db-security-tests.live.csv`). The script `ROOTS-A_I_M2_Security_Tests.sql` is self-contained: ROOTS can run it in the Supabase SQL Editor with no set-up. It creates temporary test accounts, runs every probe as that role exactly as the Supabase API does, rolls every probe back, removes the test accounts and prints one row per test.

Group	Tests	What it proves
Positive controls	6	Permitted access works, so no PASS can come from an empty table
Anonymous	5	No access without sign-in
Cross-user	12	Participant B cannot read or change A's assessments, answers, scores, report, consents or profile
Browser role	16	Participants cannot create/change scores, reports, audit records or roles; cannot submit directly, edit after submission, re-open, delete, change versions or forge the source version
Privacy	3	Answer-bearing columns (calculation trace, report content) are unreadable through the API, even by their owner
Cross-role	10	Admin cannot read raw answers or change scores; expired admin access sees nothing
Integrity	7	No role, including the server, can change stored scores, submitted answers, reports or source versions, or delete audit records
Schema	2	Current schema in place; no write privilege on server-only tables
RLS enabled	11	Row Level Security on every application table
Clean-up	1	Test data removed

1. **API negative tests — 50/50 PASS against the live deployment** (`evidence/api-negative-tests.html`). No session and forged sessions (forged cookie, forged service-role token) are refused on every protected route; malformed IDs, bodies, idempotency keys and injected values are rejected; unsupported methods are refused; protected pages expose no data; security headers are present. Reproduce: `BASE_URL=<deployment> npm run evidence:api`.
2. **Input-bypass tests — 23/23 PASS** (`tests/security/bypass.test.ts`). Invalid inputs cannot bypass the canonical controls: 14 kinds of tampered stored answers are blocked at submission; only the 40 scoring items can influence scores; forged options and raw point values cannot be scored; no endpoint accepts scores or reports from a client.

Note on C-06. C-06 §1 refers to "RLS policies + 12 negative tests". The technical bundle containing those tests was not included in the files received. The 73 tests above cover every requirement C-06 names (cross-user and cross-role access blocked; browser roles cannot create scores, reports or audit records). If the original 12 tests are available, please send them and we will run them as well.

10. Controlled technical source

`python scripts/canonical/generate.py --check` regenerates the application data from the two XLSX workbooks and fails on any difference. Each generated file records the workbook name, document version and SHA-256 (`evidence/source-check.txt`). The only transformations are type normalisation and parsing of the golden-test cells, which the workbook stores as JSON text.

Workbook	SHA-256
C-01 v1.0.1 CORRECTED	8c9c9e601d882e0f694c43a6c5d6cc5be5c8007f870283da7804ce3d32b7444c
C-02 v1.0.1 CORRECTED	12dc3a66a68c4afa9497101959408984ba653b3bb6a099624502429f1e66a028

11. GitHub

The current working codebase is pushed to the ROOTS-owned repository `ROOTS-AI-Health-Systems/demo-repository` , branch `main` , commit `4056363` . As agreed, interconnected later work is present in the repository; M2 acceptance covers only the contractual M2 scope above.

Reproducing the results

```
npm ci
npm test # 252 automated tests
npm run evidence:golden # Golden Test evidence report
python scripts/canonical/generate.py --check
BASE_URL=<deployment> npm run evidence:api # API negative tests
```

Database: run `supabase/roots_ai_complete.sql` , then `docs/m2/ROOTS-AI_M2_Security_Tests.sql` , in the Supabase SQL Editor. Every row must read PASS.

Package contents

File	Content
ROOTS-AI_M2_Completion_Package.pdf	This document
ROOTS-AI_M2_Security_Tests.sql	Self-contained database security test script
evidence/golden-tests.pdf · .html · .csv · .json	30 Golden Tests, case by case
evidence/db-security-tests.live.csv	73/73 database security results (live)
evidence/api-negative-tests.html · .json	50/50 API negative results (live deployment)
evidence/automated-tests.txt	Full automated test run (252)
evidence/source-check.txt	Generated data vs controlled workbooks
supabase/roots_ai_complete.sql	Complete database script (schema, RLS, integrity rules)

Points for ROOTS to confirm

1. questionnaire_version is stored as **1.0.0**, as the C-01 v1.0.1 CORRECTED workbook states it, with the controlled source C-01 v1.0.1 CORRECTED recorded alongside (§7).
2. The Recovery Potential band rule for one-decimal values (§8).
3. The original C-06 "12 negative tests", if available (§9).